CS 4400: Artificial Intelligence
Beyond Classical Search

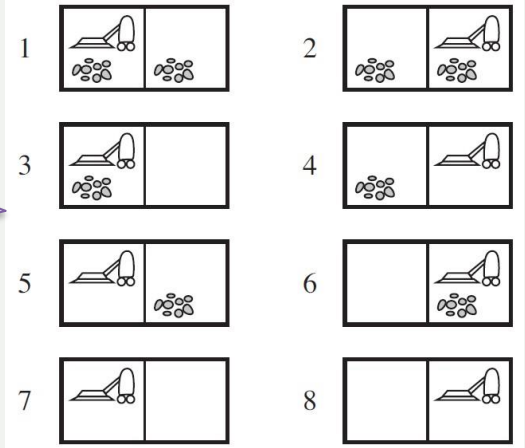
Contents

1. Nondeterministic environment
   - Erratic vacuum world
   - AND-OR search trees
   - Slippery vacuum world
2. Unobservable environment
   - Belief state and belief state-space
3. Partially observable environment
   - AND-OR search
4. Recursive state estimation

1/4. Value of percepts

- In fully observable, deterministic environment
  - Assume the initial state is known to the agent
  - Agent can determine exactly which states it is in after taking any sequence of actions
  - Percepts provide no new information
- When does an agent need percepts?
  - When initial state is not known
  - When environment is non-deterministic (=stochastic)
  - Ex. erratic vacuum world, and slippery vacuum world

Erratic vacuum world



The eight possible states of the vacuum world; states 7 and 8 are goal states

- *Suck* at square A
  - When A has dirt, it removes dirt, sometimes removes dirt in B as well
  - When A is clean, it sometimes deposits dirt
- Result(a,s) → Results(a,s)
- Contingency plan
  - [*Suck*, if *state=5* then [*Right,Suck*] else […]]
  - Need a tree, rather than a single sequence to solve the problem

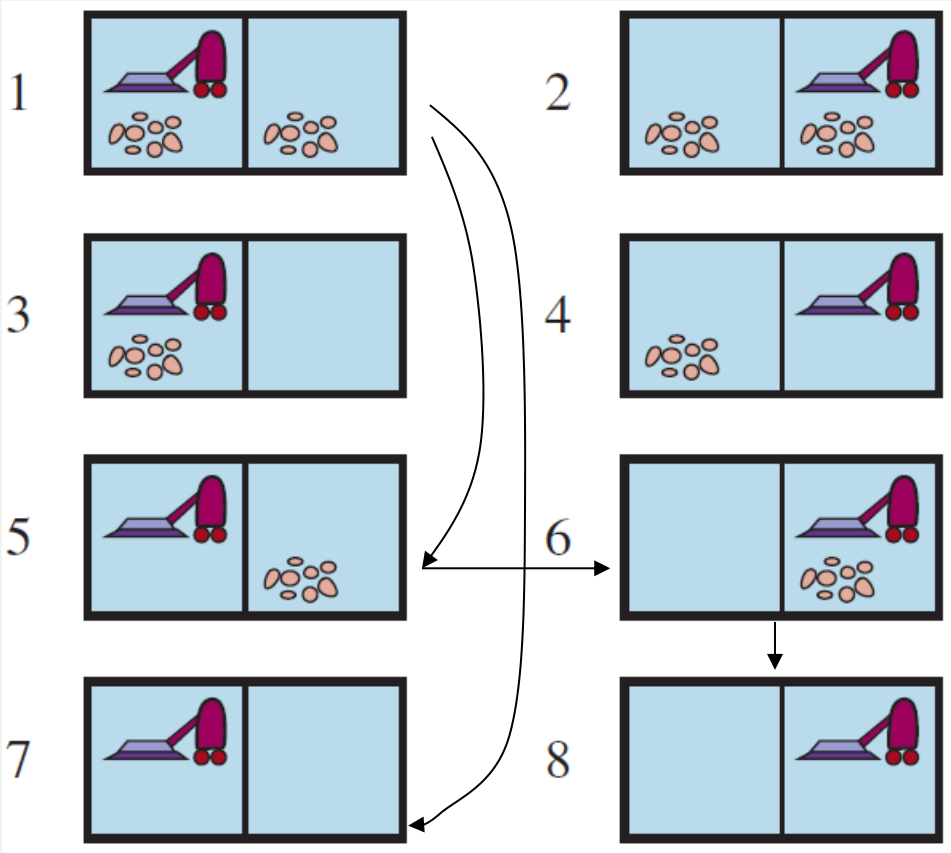
This allows the selection of actions based on contingencies arising during execution

Erratic vacuum world

No single sequence of actions solves the problem.

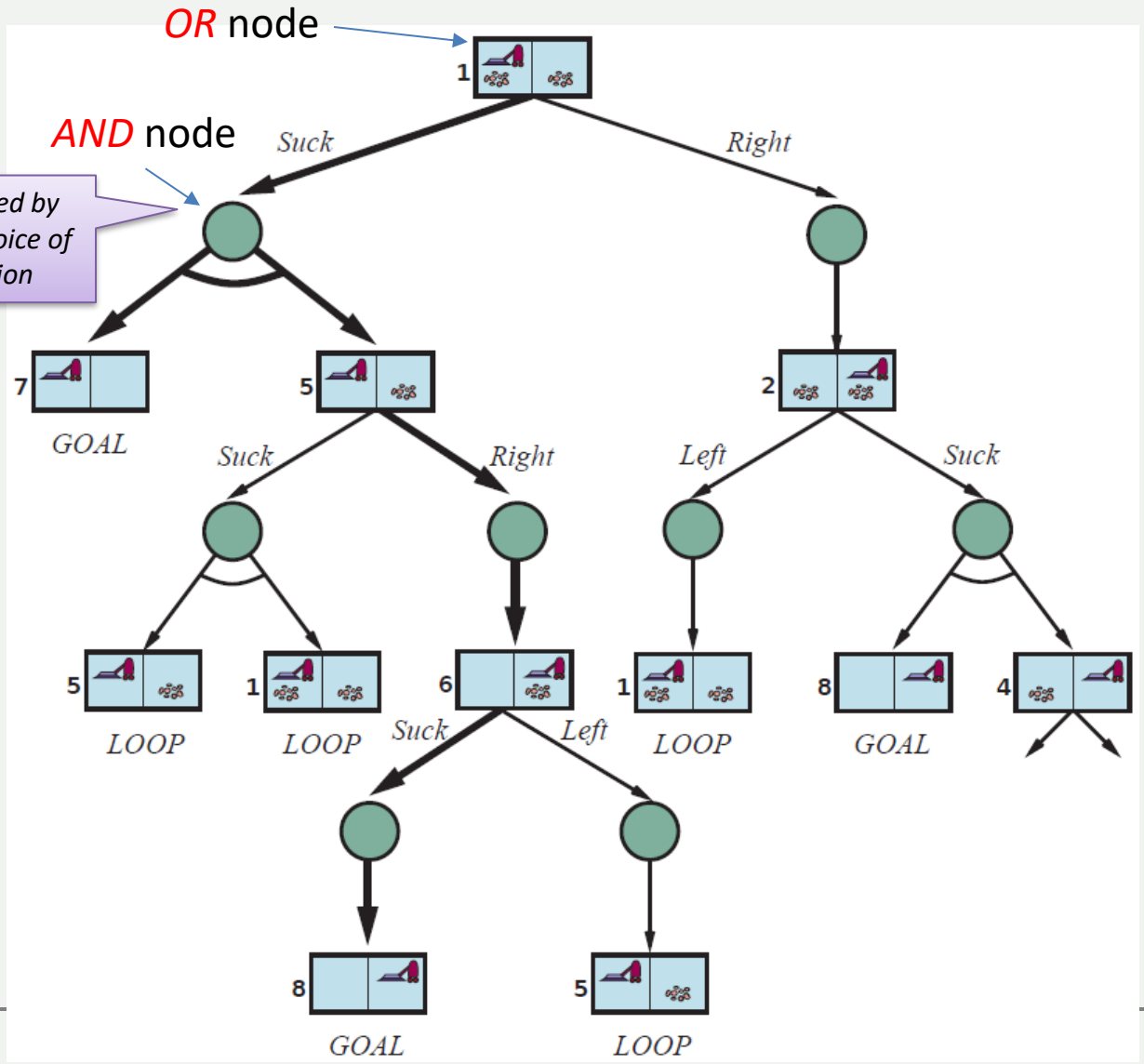
Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck,* **if** State = 5 **then**
[*Right, Suck*] **else** [ ]]



Solution is a tree – of a different character!

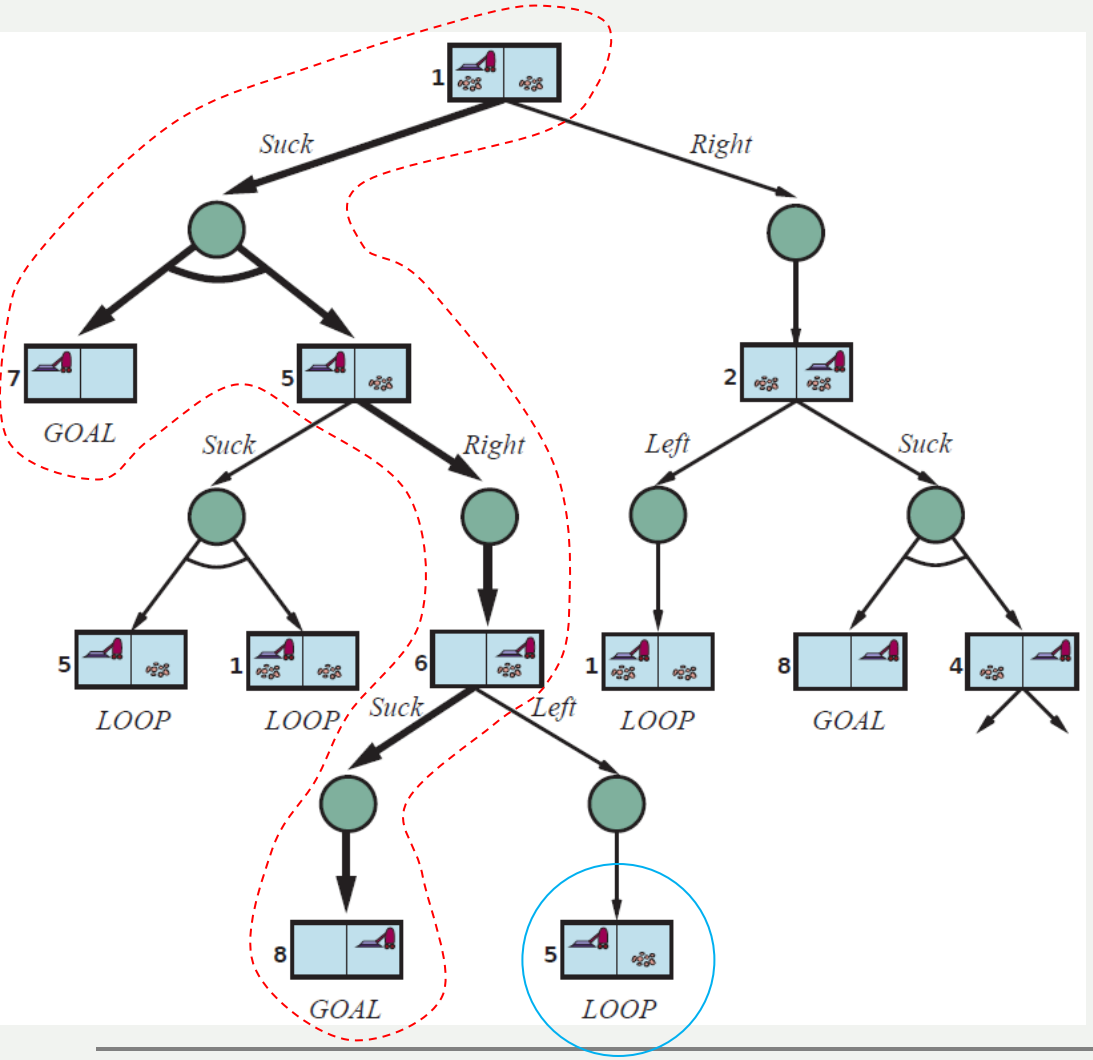
AND-OR search trees



AND-OR search

- OR nodes = state nodes
  - Branching by different actions
- AND nodes
  - Branching by nondeterministic environment
- Solution in AND-OR tree is a subtree which
  1) Has a goal node at every leaf
  2) Specifies one action at each of its OR nodes
  3) Includes every outcome at its AND nodes

Example of a (Partial) Tree



A *solution* is a connected portion of the AND-OR tree such that

- its root is the tree's root;
- every OR node has exactly one child (i.e., one of the actions);
- every AND node has all children (possible outcomes) from the corresponding action;
- all the leaves are goal nodes.

Recursive DFS for AND-OR graph search

function AND-OR-GRAPH-SEARCH(*problem*) **returns** *a conditional plan, or failure*
  OR-SEARCH(*problem*.INITIAL-STATE, *problem*, [ ])

---

function OR-SEARCH(*state*, *problem*, *path*) **returns** *a conditional plan, or failure*
  if *problem*.GOAL-TEST(*state*) **then return** the empty plan
  if *state* is on *path* **then return** *failure*
  for each *action* **in** *problem*.ACTIONS(*state*) **do**
    $plan \leftarrow$ AND-SEARCH(RESULTS(*state*, *action*), *problem*, [*state* | *path*])
    if $plan \neq failure$ **then return** [*action* | *plan*]
  return *failure*

---

function AND-SEARCH(*states*, *problem*, *path*) **returns** *a conditional plan, or failure*
  for each s_i **in** *states* **do**
    $plan_i \leftarrow$ OR-SEARCH(s_i, *problem*, *path*)
    if $plan_i = failure$ **then return** *failure*
  return [**if** s_1 **then** $plan_1$ **else if** s_2 **then** $plan_2$ **else** . . . **if** s_{n-1} **then** $plan_{n-1}$ **else** $plan_n$]

For simplicity, ignore path cost.

Recursive DFS for AND-OR graph search

```
function AND-OR-GRAPH-SEARCH(problem) returns a conditional plan, or failure
    OR-SEARCH(problem.INITIAL-STATE, problem, [ ])

function OR-SEARCH(state, problem, path) returns a conditional plan, or failure
    if problem.GOAL-TEST(state) then return
    if state is on path then return failure
    for each action in problem.ACTIONS(state
        plan ← AND-SEARCH(RESULTS(state,
        if plan ≠ failure then return [action |
    return failure

function AND-SEARCH(states, problem, path) returns a conditional plan, or failure
    for each s_i in states do
        plan_i ← OR-SEARCH(s_i, problem, path)
        if plan_i = failure then return failure
    return [if s_1 then plan_1 else if s_2 then plan_2 else ... if s_{n-1} then plan_{n-1} else plan_n]
```

*If the current state is identical to a state on the path from the root, it returns failure.

➔ Does not mean that there is no solution from the current state; it simply means that if there is noncyclic solution, it must be reachable from the earlier incarnation of the current state, so the new incarnation can be discarded

For simplicity, ignore path cost.

Slippery vacuum world

Movement actions sometimes fail, leaving the agent in the same location.

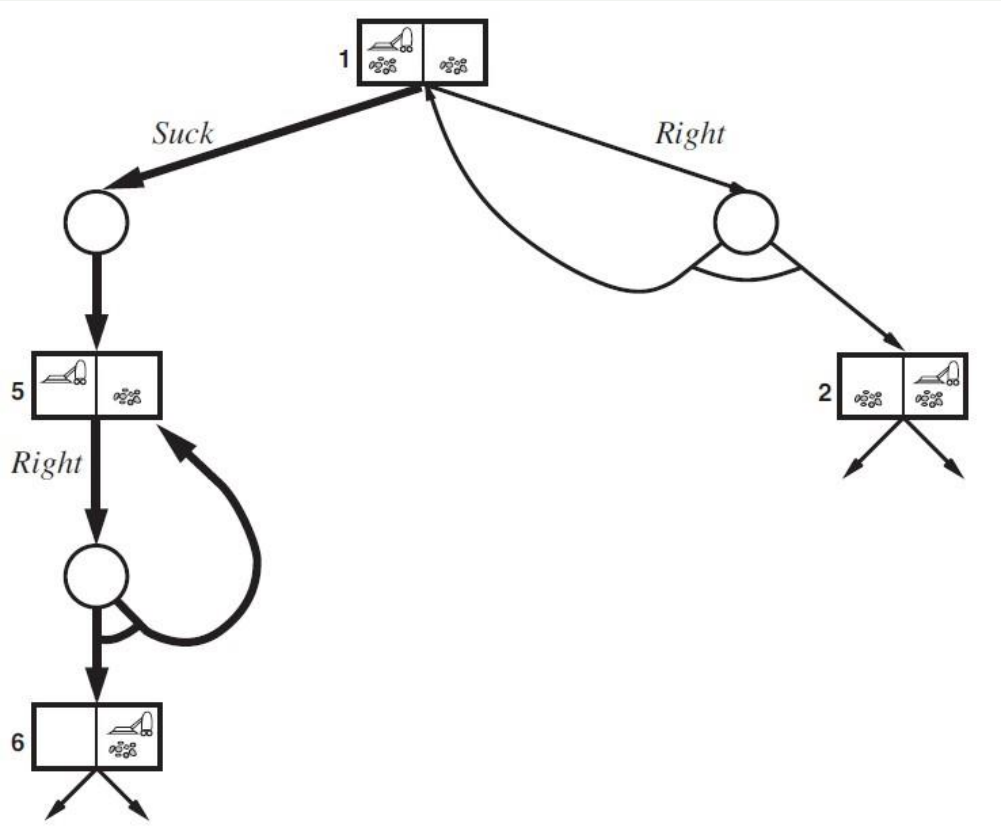
- "Suck" works fine, but "Left/Right" has no effect from time to time
  - state 5 → state 5
  - cyclic

    solution:

    while *state=5*

    do *right*
  - AND-OR search

  can be modified



2/4. Unobservable environment

- No observation (a.k.a. sensorless problem)
  - To avoid costly observation: broad-spectrum antibiotics instead of expensive blood test
- Sensorless vacuum world
  - Deterministic environment (non-erratic, non-slippery)
  - Doesn't know if it's in square A or B
  - Cannot sense dirt
  - Agent's precepts provide no information at all.
- Can an agent solve sensorless problems?
  - Yes, for vacuum: [*Right, Suck, Left, Suck*]
  - No, if it's a 8-puzzle.

**The agent knows the geography of its world*

Belief states

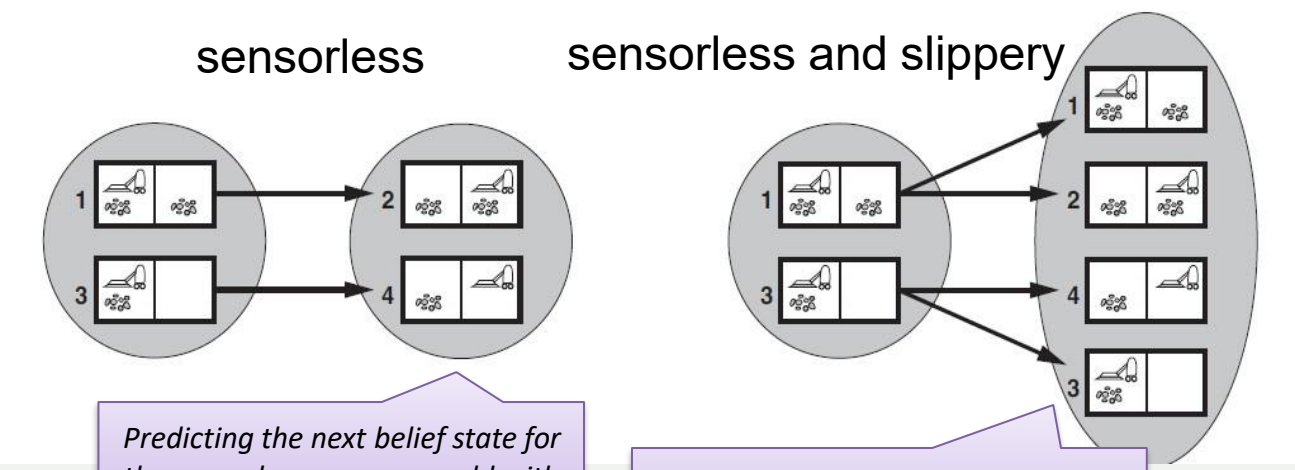
A *belief state* is a set of physical states believed to be possible by the agent.

- Belief states vs singleton state
  - Any possible subset of the powerset: 2^N
    - state = {1}, {2}, ... , {N}
    - belief state = {1},...,{N}, {1,2},...,{1,N}, {1,2,3}, ...,..., {1,2,....,N}
- Why use belief states?
  - Can handle sensorless problem
    - Agent can fully observe its own belief state !
    - Can find a solution as a sequence of actions
  - Can also handle non-deterministic problem
    - No contingency plan needed

Belief states space

Typically the set of all states in the underlying physical problem.

- Initial belief state
  - Unknown initial state → *known* belief states {1,…,8}
- Actions: two definitions
  - Union: $ACTIONS^*(b) = \cup_{s \in b} ACTIONS(s)$
  - Intersection: $ACTIONS^*(b) = \cap_{s \in b} ACTIONS(s)$
- Transition: $b' = RESULT^*(b, a) = \{s' : s' = RESULT(s, a), s \in b\}$



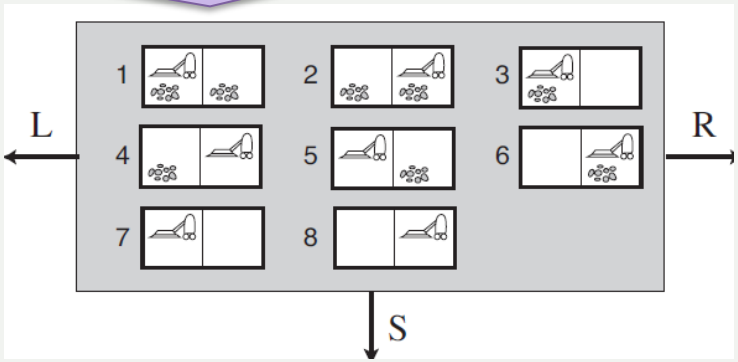
Predicting the next belief state for the sensorless vacuum world with a deterministic action, Right

Prediction for the same belief state and action in the slippery version of the sensorless vacuum world

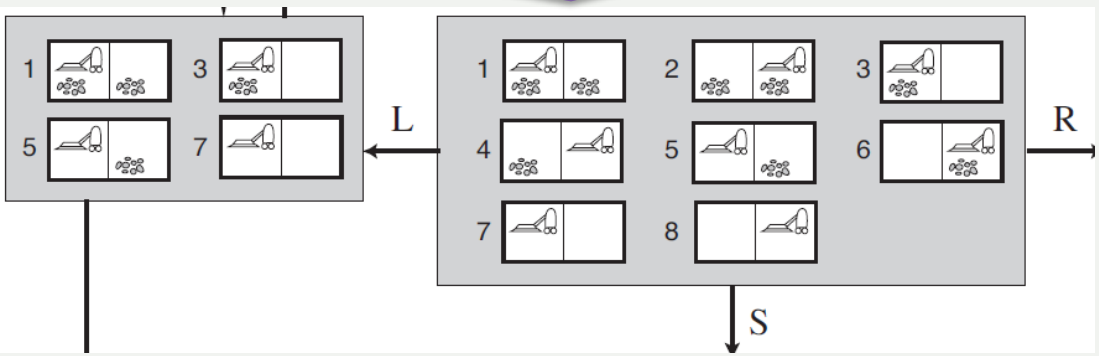
- Results of the transition in belief states will be called *prediction*
- Goal test: all singleton goal states and their combinations
  - Ex. if {1} and {2} are goals, then {1}, {2}, and {1,2} are belief state Goals (if all the physical states in it satisfy 'goal')
  - {1,3} is not a belief state goal: agent may accidentally achieve the goal {1} earlier, but it won't know that it has done so.
- Path cost
  - Difficult to compute if the cost of each singleton action is different
  - Assume unit cost for the belief state actions (*for now, we assume that the cost of an action is the same in all states and so can be transferred directly from the underlying physical problem.)*

Belief state-space (sensorless, deterministic)

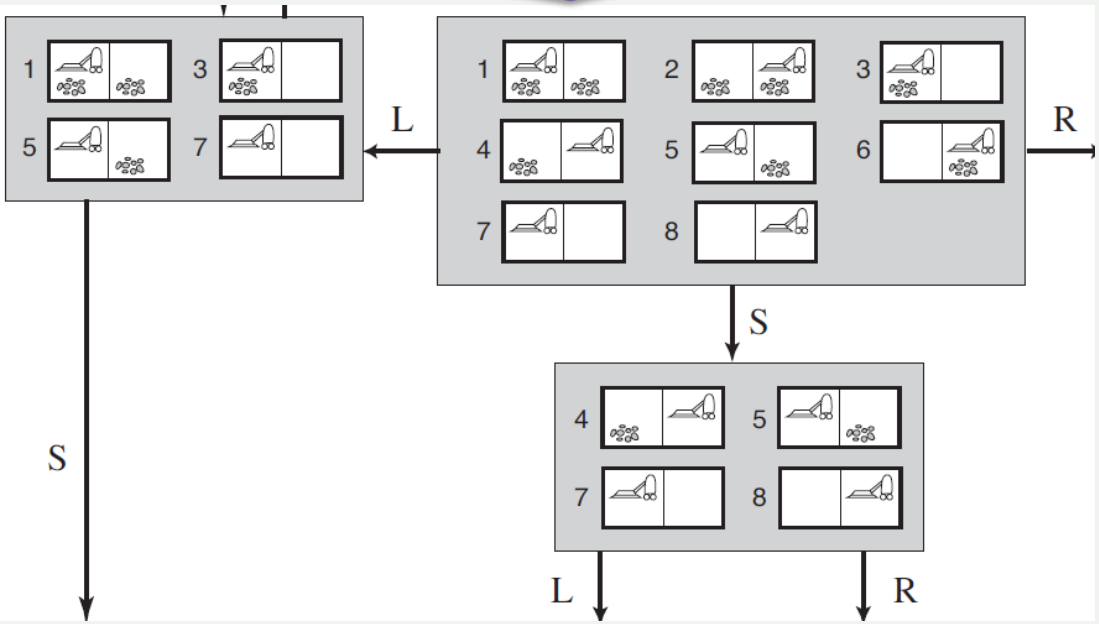
The reachable portion of the belief-state space for the deteterministic, sensorless vacuum world. Each shaded box corresponds to a single belief sate. At any given point, the agent is in a particular belief state but not know which physical state it is in.



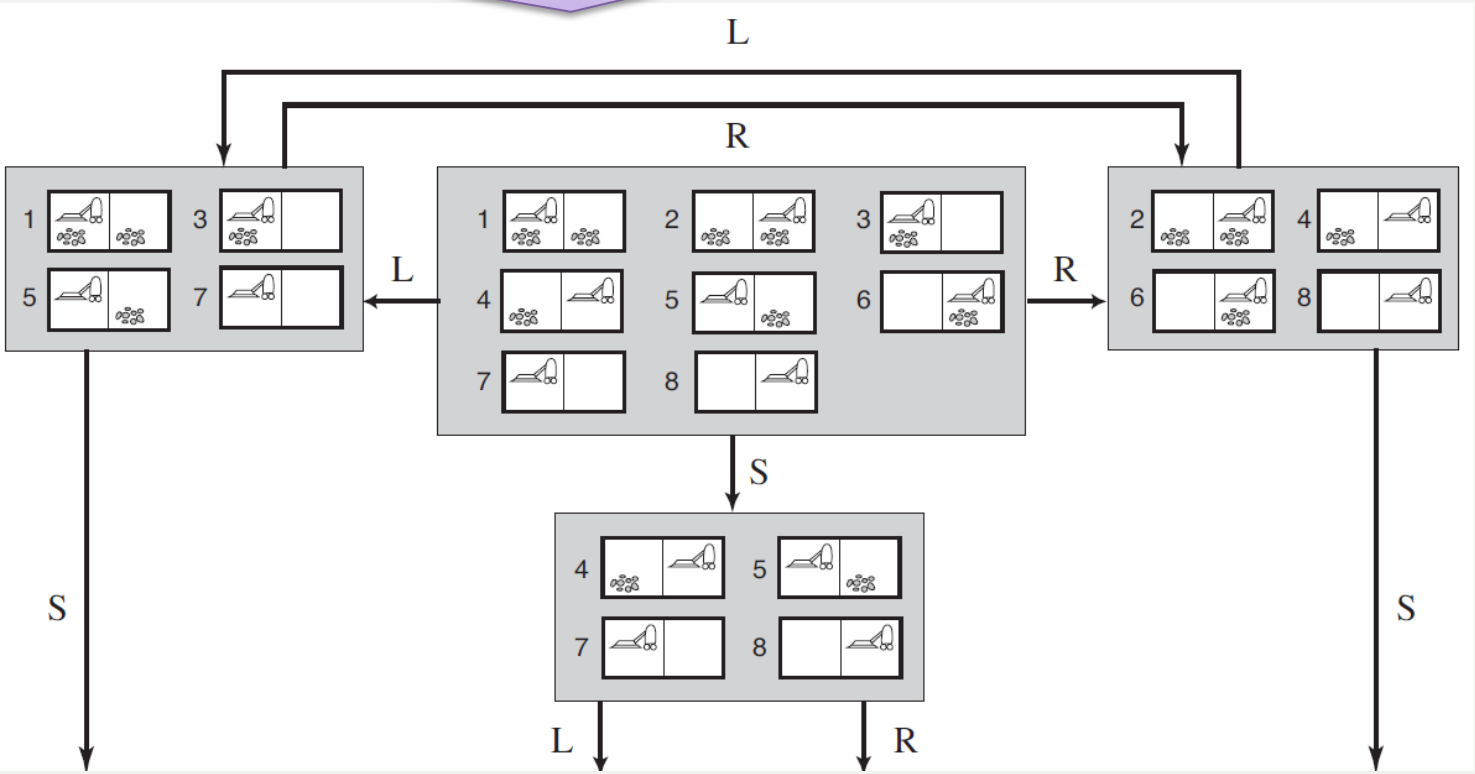
The reachable portion of the belief-state space for the deteterministic, sensorless vacuum world. Each shaded box corresponds to a single belief sate. At any given point, the agent is in a particular belief state but not know which physical state it is in.



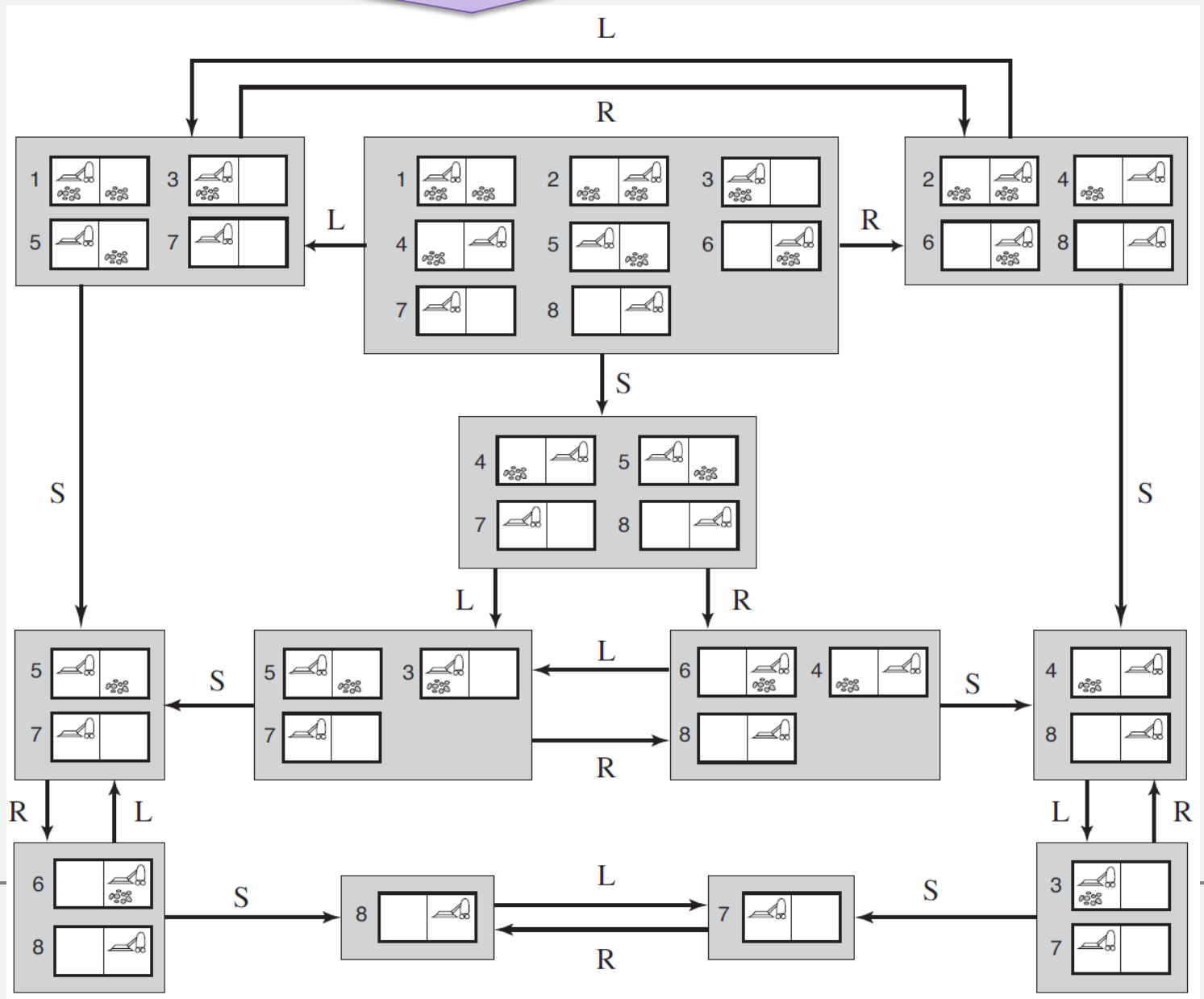
The reachable portion of the belief-state space for the deteterministic, sensorless vacuum world. Each shaded box corresponds to a single belief sate. At any given point, the agent is in a particular belief state but not know which physical state it is in.



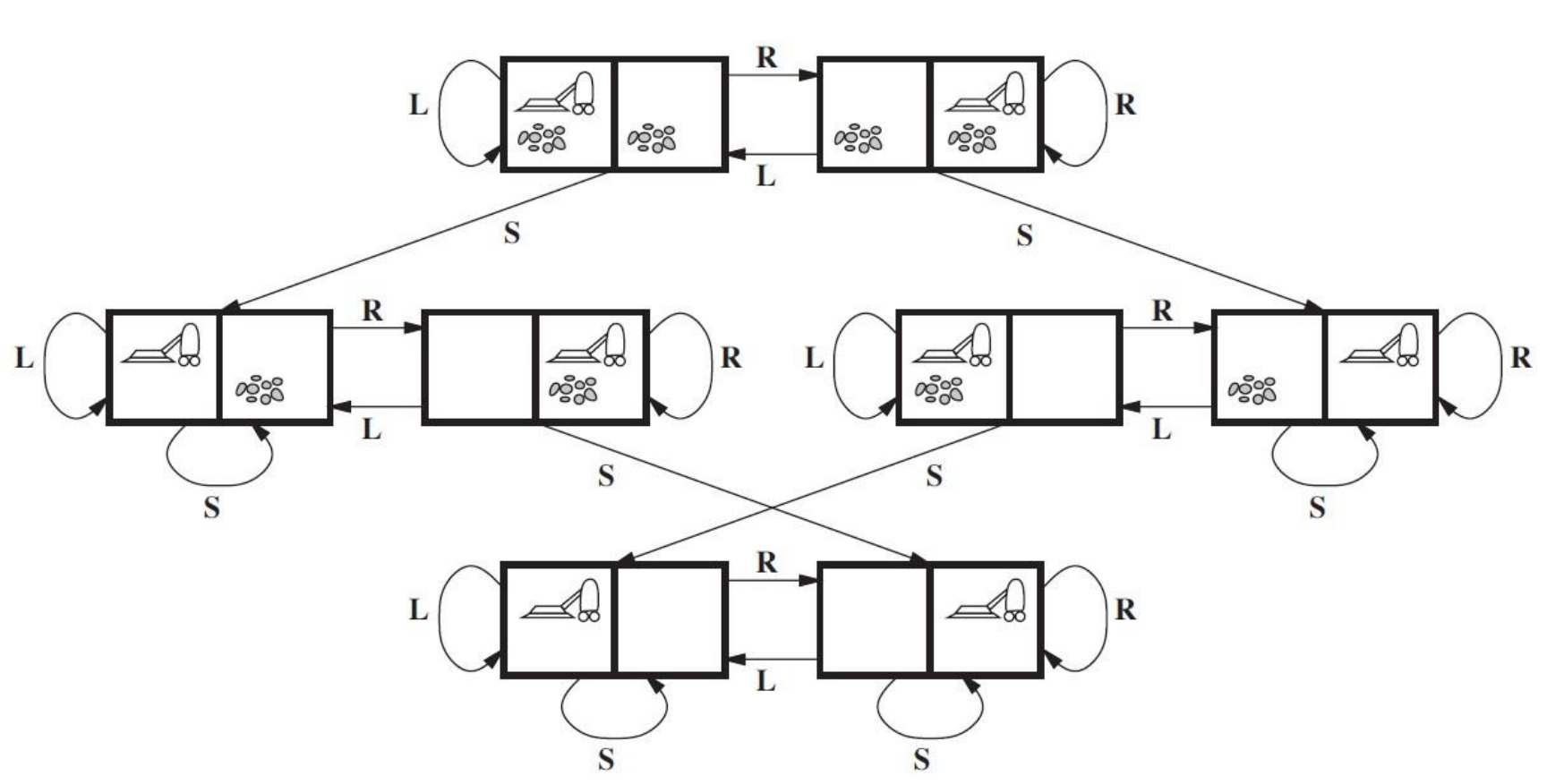
The reachable portion of the belief-state space for the deteterministic, sensorless vacuum world. Each shaded box corresponds to a single belief sate. At any given point, the agent is in a particular belief state but not know which physical state it is in.



The reachable portion of the belief-state space for the deteterministic, sensorless vacuum world. Each shaded box corresponds to a single belief sate. At any given point, the agent is in a particular belief state but not know which physical state it is in.



Cf. singleton state space



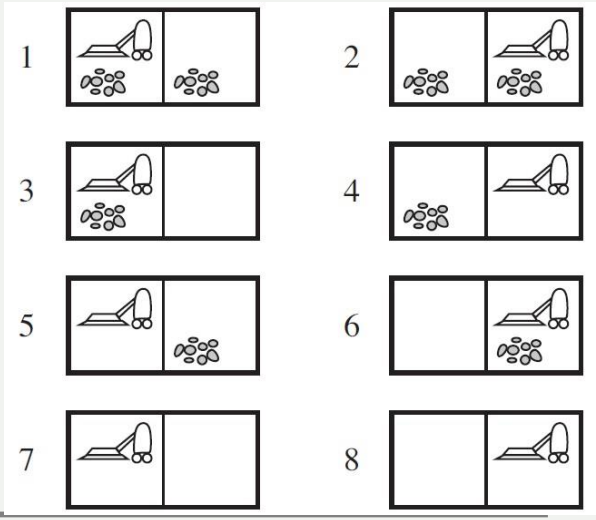
3/4. Partially observable environment

- Local-sensing vacuum agent
  - PERCEPT(1) = [*A,Dirty*]
  - Full observation:
    - PERCEPT(1) = [A, A is dirty and B is clean]
  - Suppose initial percept = [*A,Dirty*]

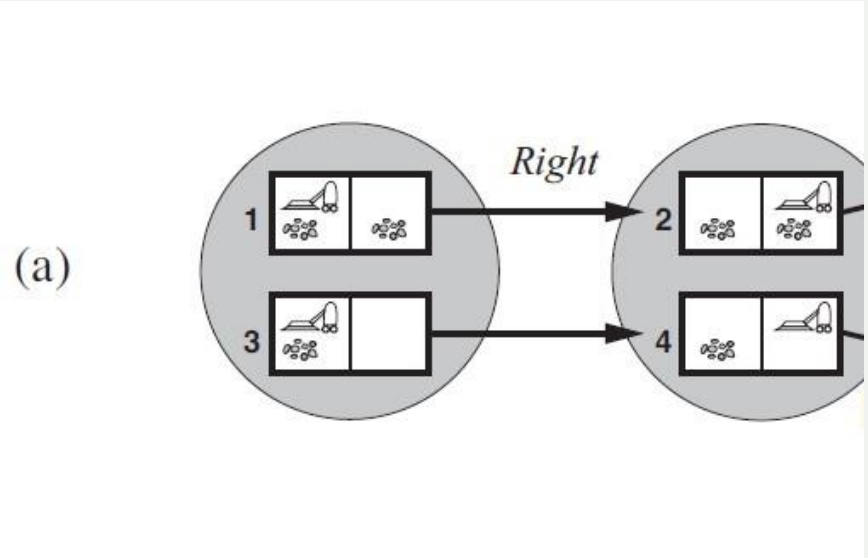
    → state = 1 or 3

    → belief state = {1,3}
  - Now take action *Right*
    - You can assume deterministic or non-deterministic environment (e.g., slippery vacuum)

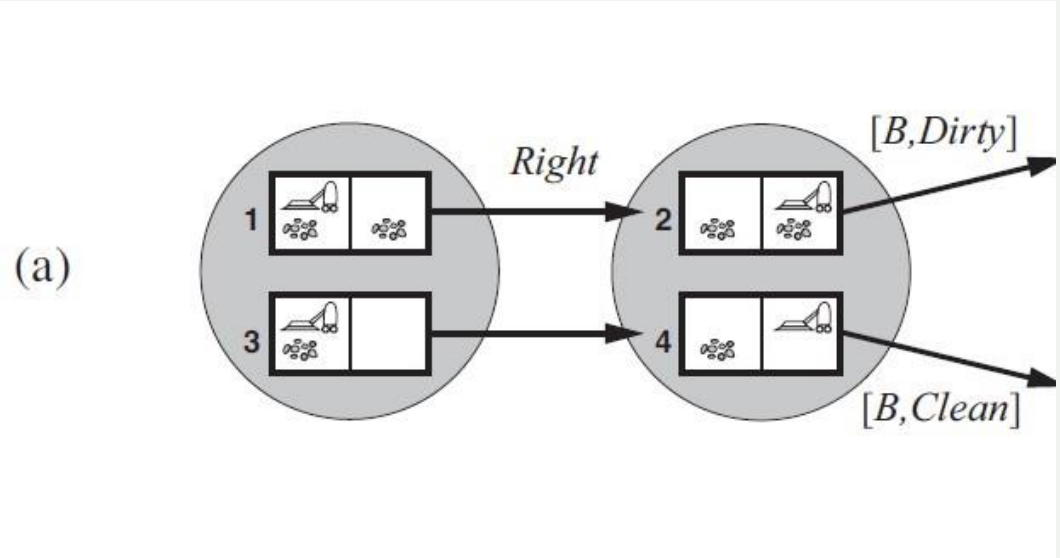
The agent may not know the current state in a partially observable environment.



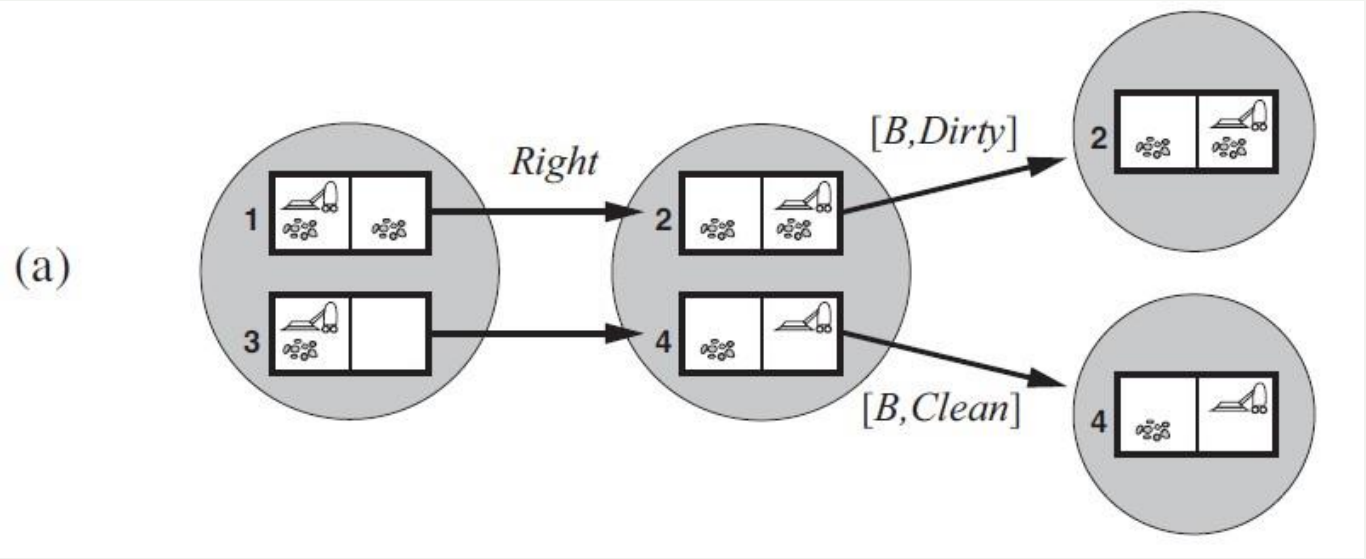
- Deterministic world: predict



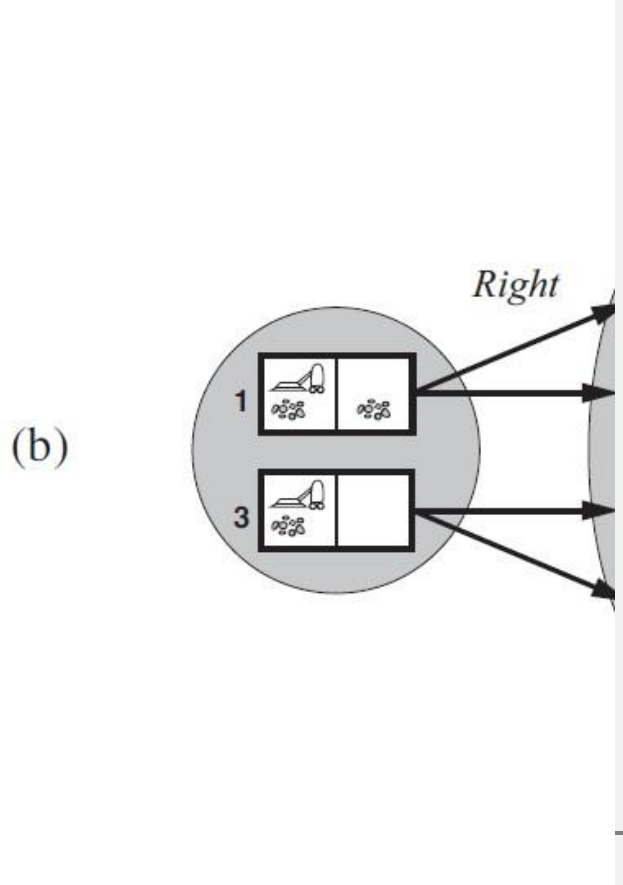
- Deterministic world: observation prediction



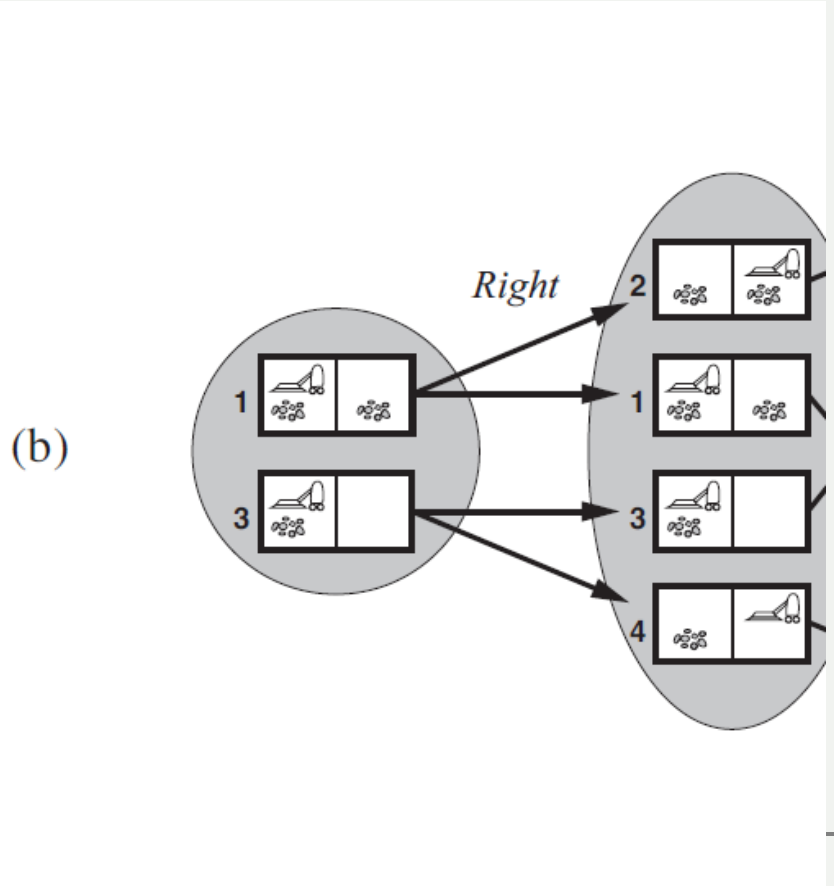
- Deterministic world: update



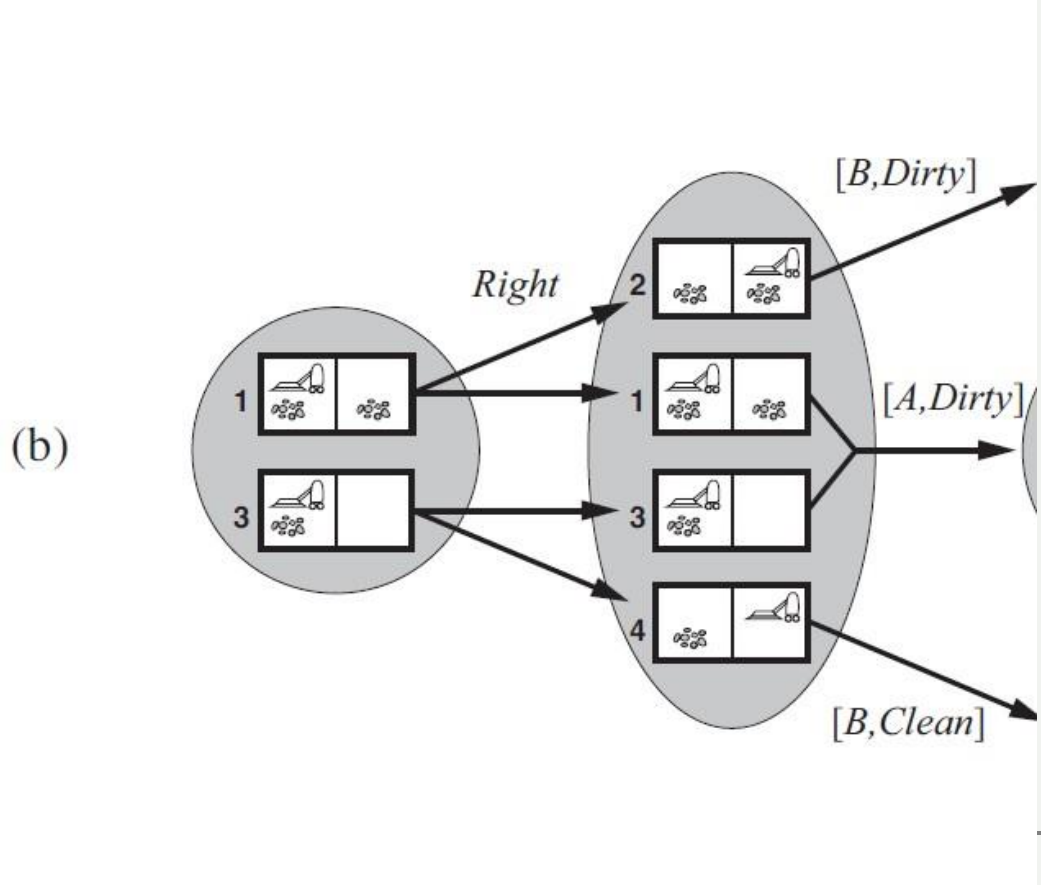
- Slippery world:



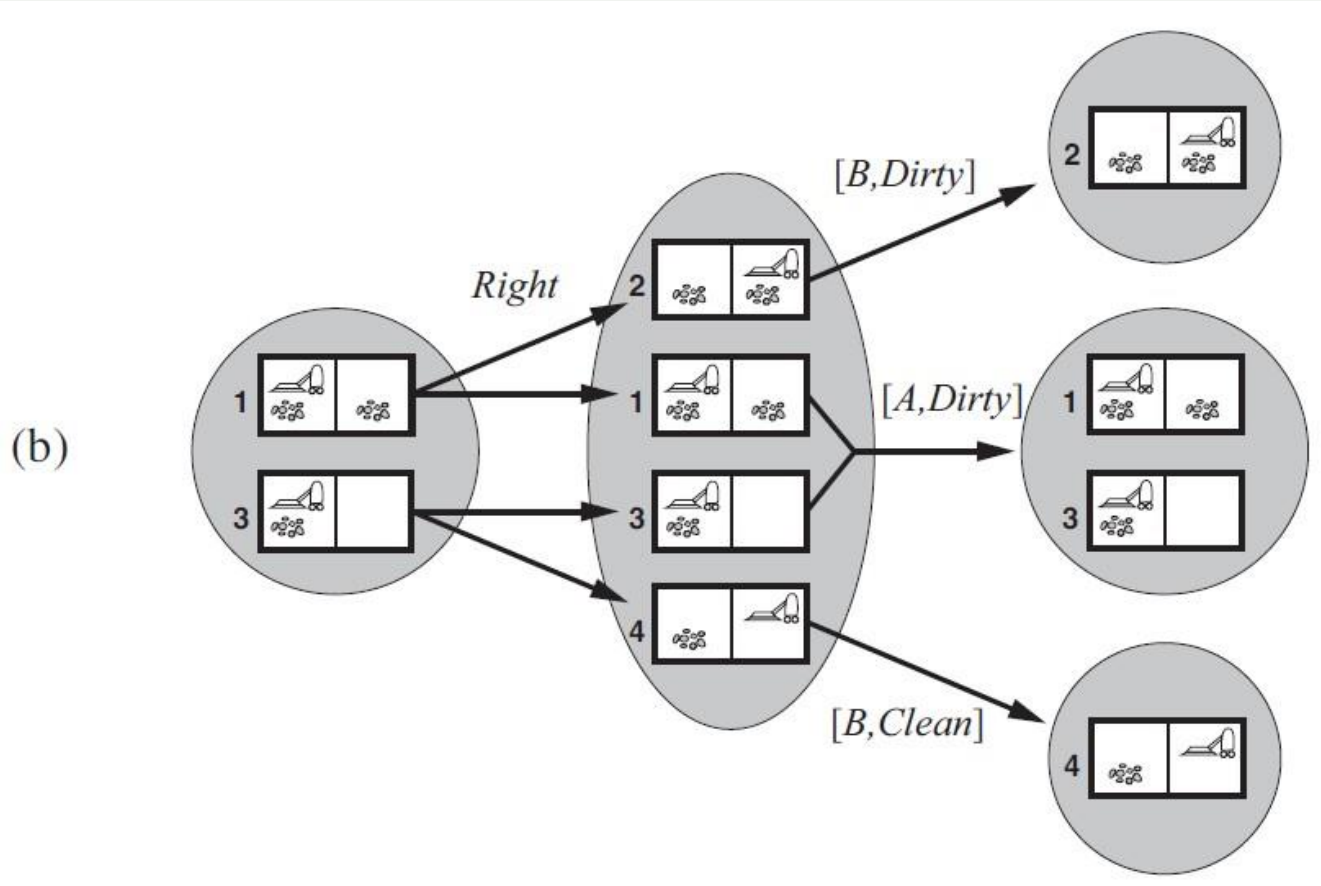
- Slippery world: prediction



- Slippery world: observation prediction



- Slippery world: update



- Prediction of belief state:
  - $b' = PREDICT^*(b, a) = \{s' : s' = RESULT(s, a), s \in b\}$
- Observation prediction

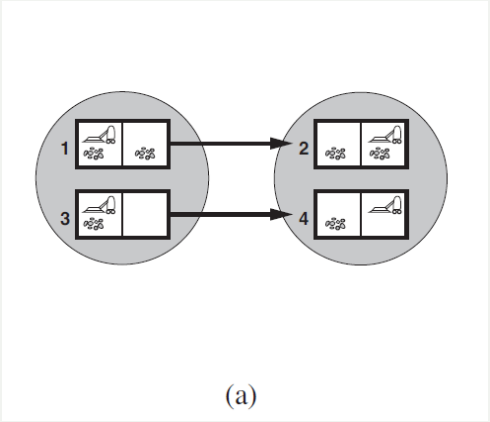
  *Determines the set of percept **o** that could be observed in the predicted belief state:*
  - $POSSIBLE\ PERCEPTS^*(b') = \{o : o = PERCEPT(s), s \in b'\}$
- Update

  Determines, for each possible percept, the belief state that would result from the percept
  - $b_o = UPDATE^*(b', a) = \{s : o = PERCEPT(s), s \in b'\}$

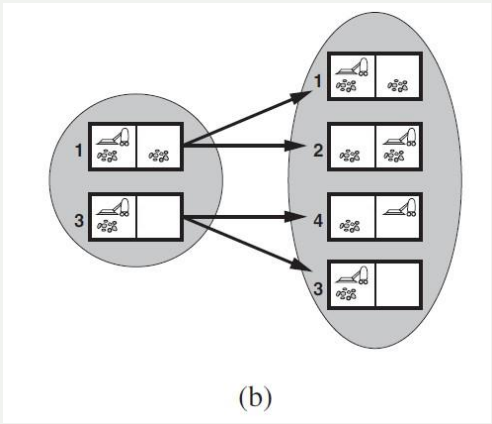
    Updated belief state

Comparison of belief state spaces

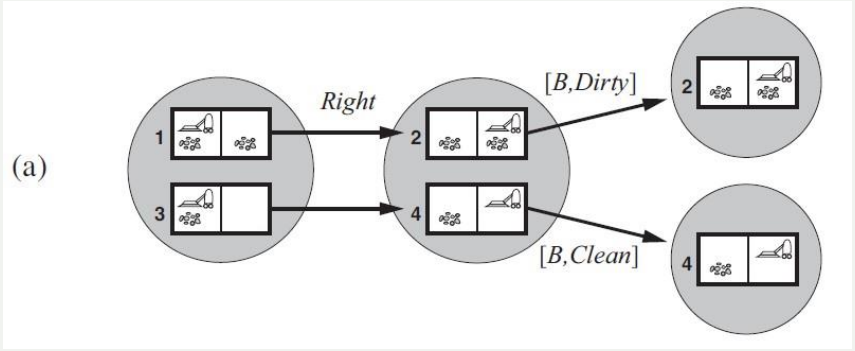
Deterministic, no obs



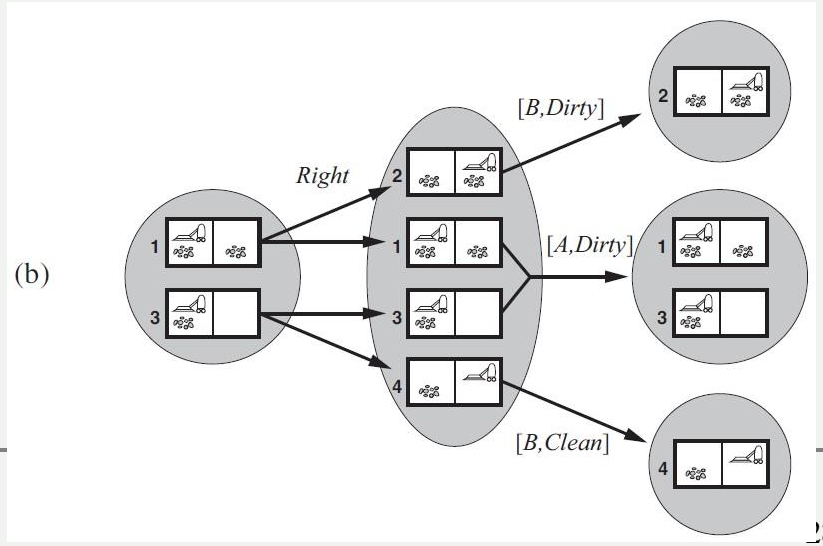
Slippery, no obs



Deterministic, partial obs



Slippery, partial obs



Comparison of belief state spaces

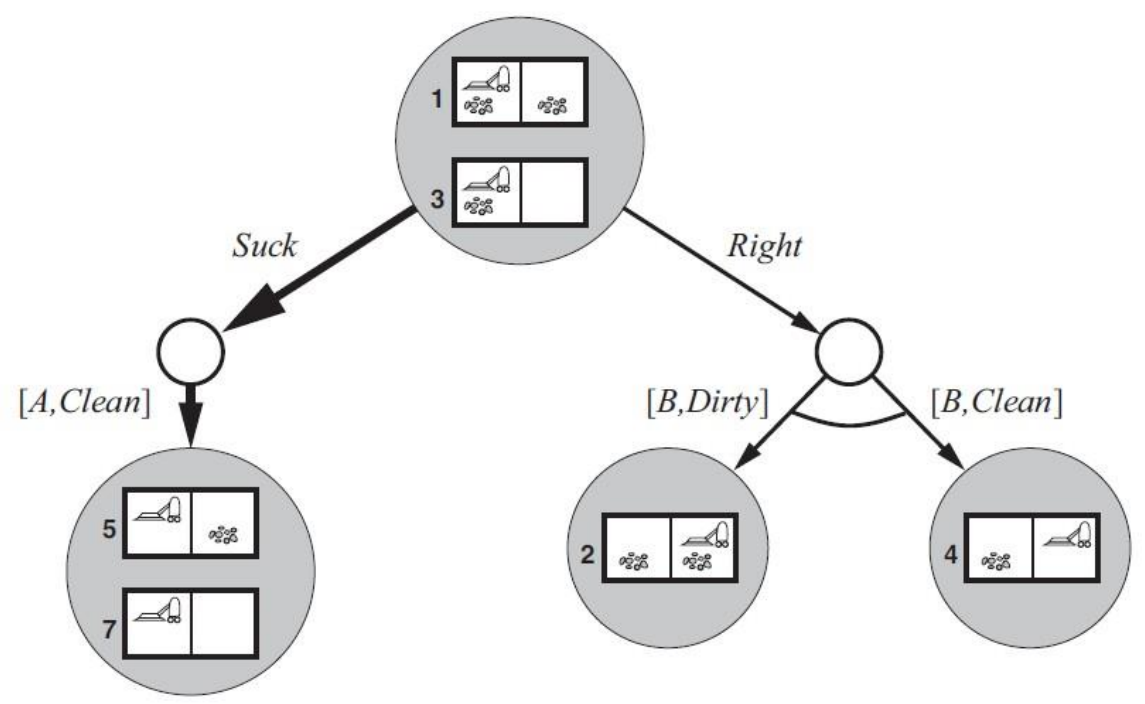
Two example of transitions in local-sensing vacuum worlds.

(a) In the deterministic world,
Right is applied in the initial belief state, resulting in a new belief state with two possible physical states; for those states, the possible percepts are [B, Dirty] and [B, Clean], leading to two belief states, each of which is a singleton.

(b) In the slippery world,
Right is applied in the initial belief state, giving a new belief state with four physical states; for those states, the possible percepts are [A, Dirty], [B, Dirty], and [B, Clean],leading to three belief states as shown.

Partially observable environment

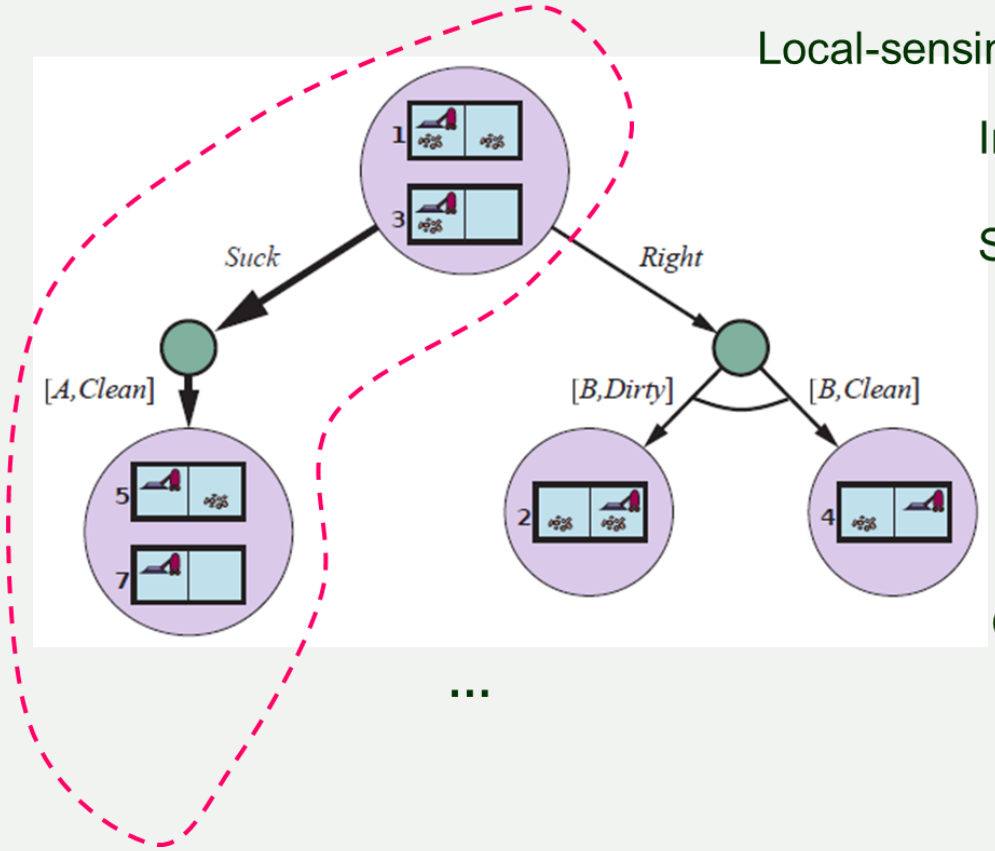
- Use AND-OR search on belief state space
  - Solution: [*Suck*, *Right*, if *Bstate* = *{6}* then *Suck* else […]]



**Part of the search tree for the local-sensing vacuum world, assuming an initial percept [A, Dirty]*
**The solution is the conditional plan that tests belief state rather than the actual state.*

Solution with Partial Observation

Apply the AND-OR search algorithm.



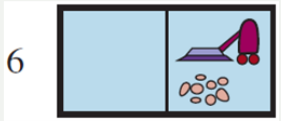
Local-sensing vacuum world

Initial percept: [*A*, *Dirty*]

Solution:

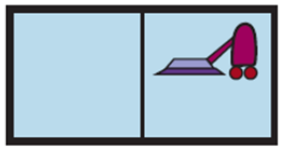
Suck
Right
if b-state={6}
then *Suck*
else []

6



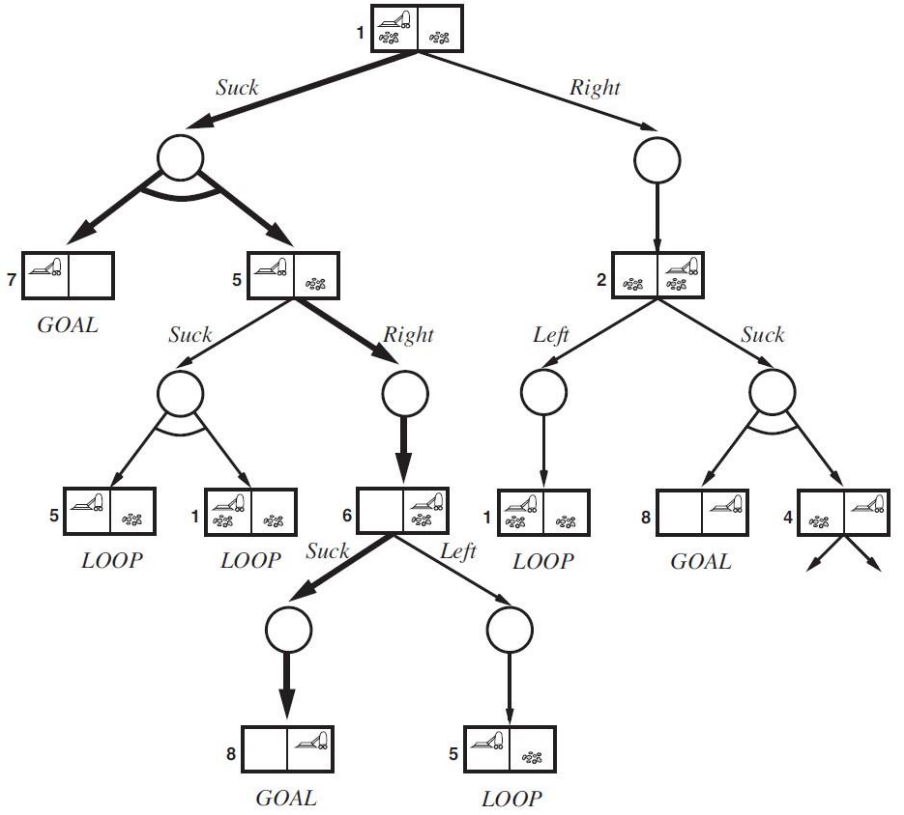
Goal:

8

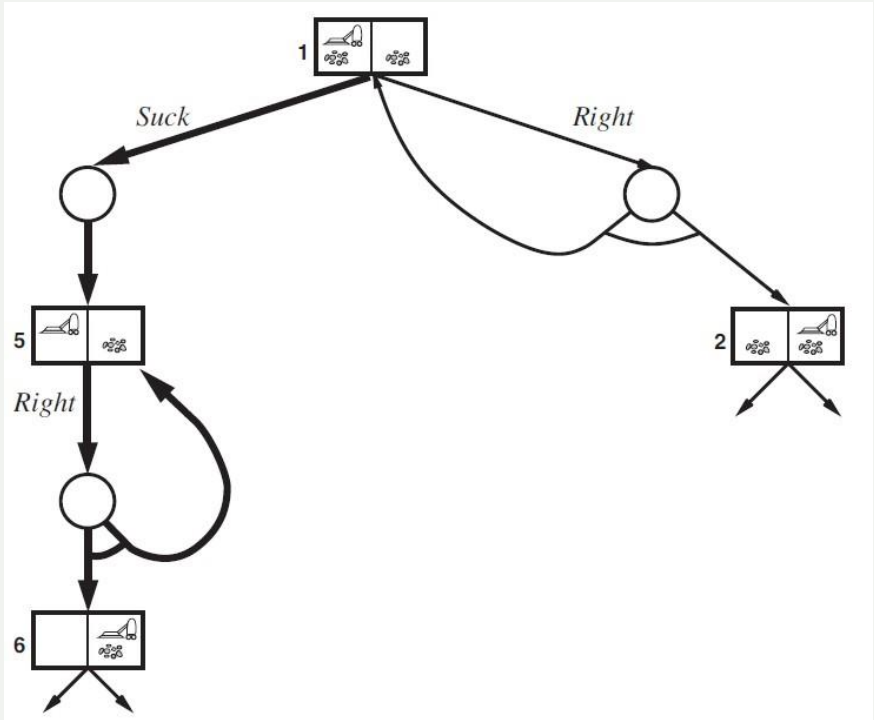


Cf. Fully-observable, stochastic

- Erratic vacuum



Slippery vacuum



Summary

- Fully-observable, deterministic
  - use uninformed/informed search
- Fully-observable, non-deterministic
  - use AND-OR search
- Unobserved, deterministic
  - use belief state + uninformed/informed search
- Partially-observable, deterministic
  - use belief state + AND-OR search
- Partially-observable, non-deterministic: same as above
  - Why? The effect of partial observability is very similar to having non-deterministic environments
  - ~~Most realistic, but more complex to find solution~~

4/4. Agent in partially observable environment

- *The design of a problem-solving agent for partially observable environments.*

➔ *The agent formulate a problem, calls a search algorithms (s.t AND-OR-GRAPH-SEARCH) to solve it, and execute the solution.*

➔ *The solution to a problem is a conditional plan rather than a sequence*

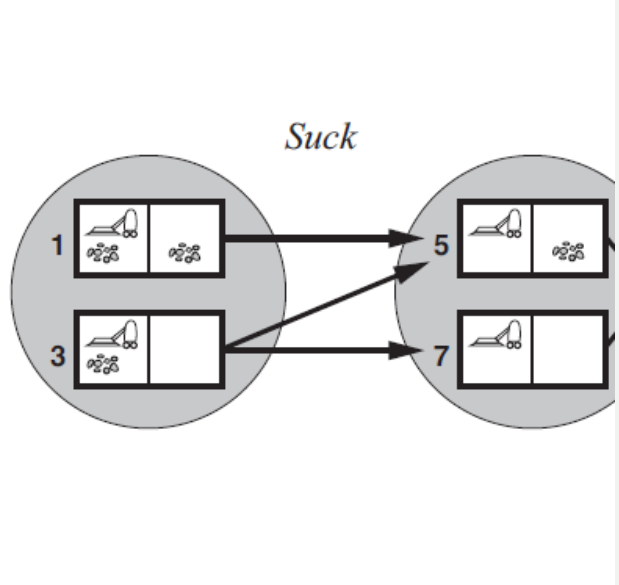
➔ *The agent will need to maintain its belief state as it performs actions and receives percepts.*

- Searching in action:
  - Agent receives one percept at a time
  - Recursively update the current belief state (*b*:state, *a*: action, *o*:percept)
  - $b_2 = UPDATE^*(PREDICT^*(b_1, a_1), o_1)$
  - $b_3 = UPDATE^*(PREDICT^*(b_2, a_2), o_2)$
  - …
- Recursive state estimation is quite common and useful
- More details in Ch.15

Example 1

*any square may become dirty at any time unless the agent is actively cleaning it at that moment

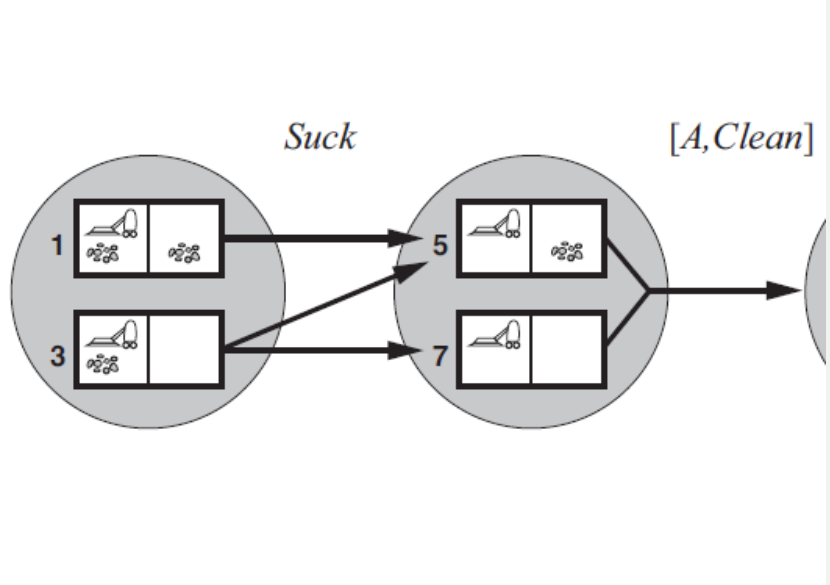
- Kindergarten vacuum world with local sensing



*Two prediction-update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing

Example 1

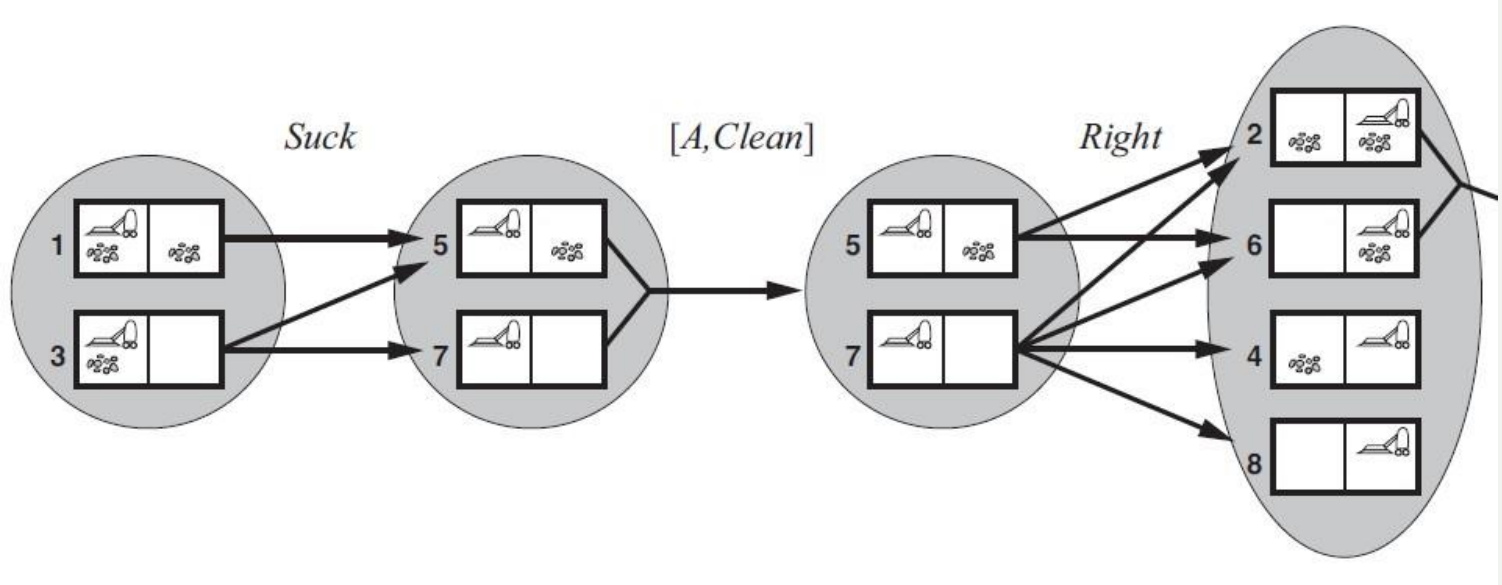
- Kindergarten vacuum world with local sensing



*Two prediction-update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing

Example 1

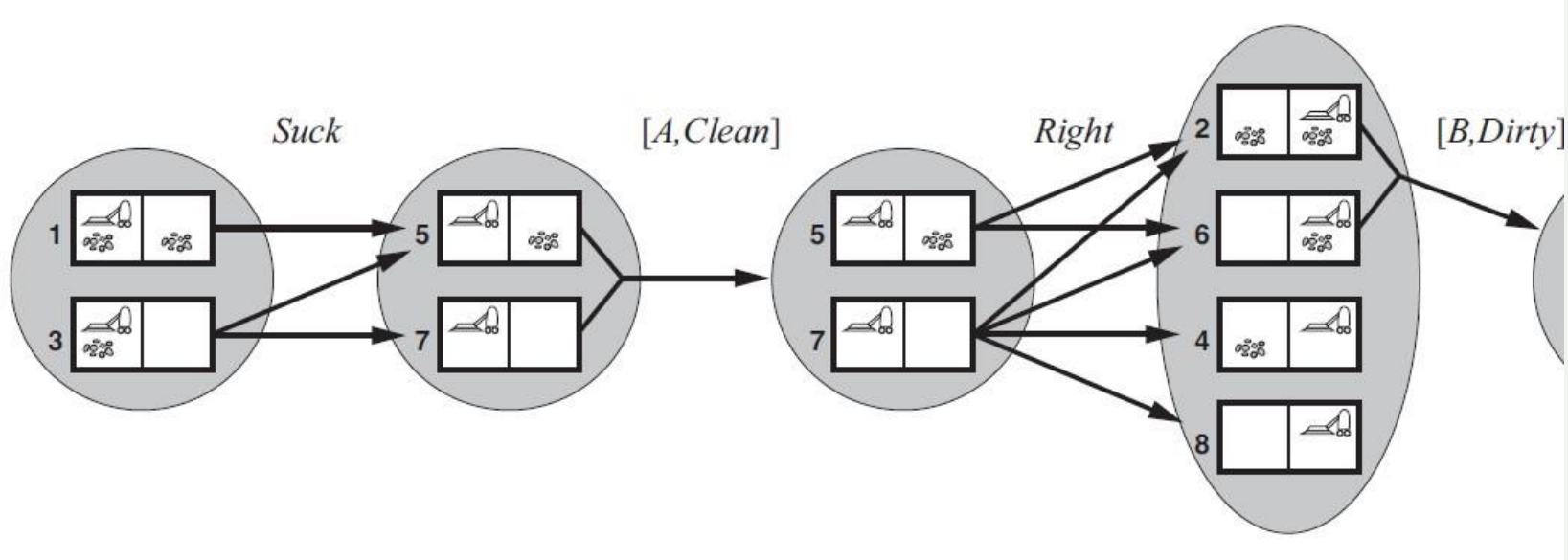
- Kindergarten vacuum world with local sensing



**Two prediction-update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing*

Example 1

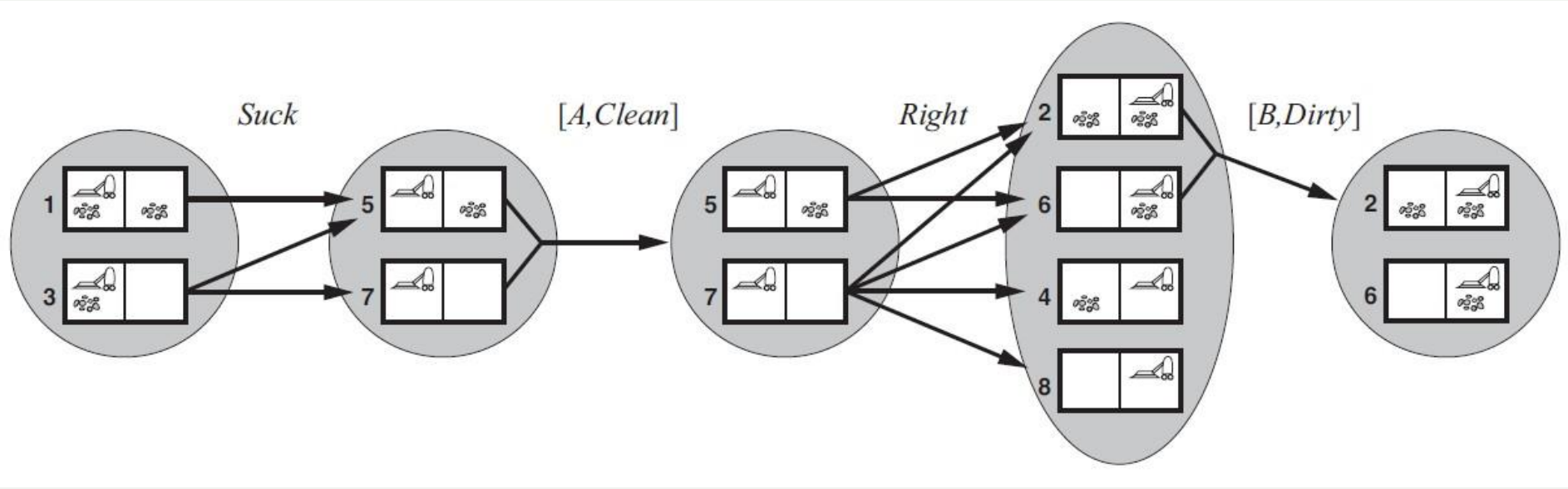
- Kindergarten vacuum world with local sensing



Two prediction-update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing

Example 1

- Kindergarten vacuum world with local sensing

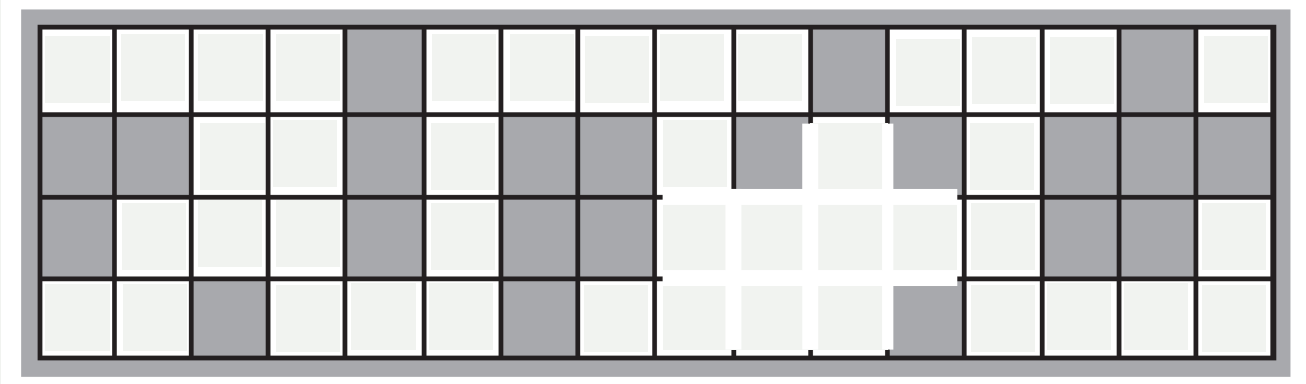


*Two prediction-update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing

Example 2: robot localization

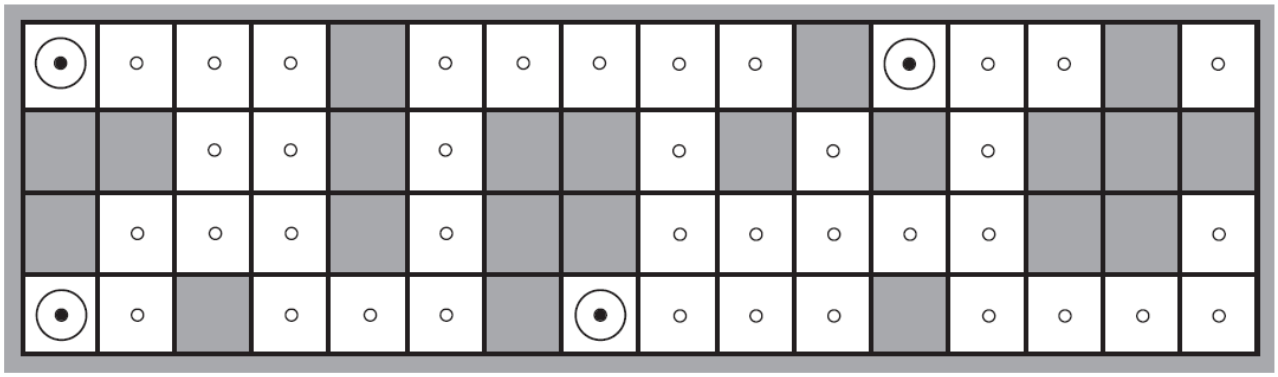
- *The robot is equipped with four sonar sensors that tell whether there is an obstacle (ex. The outer wall or a black square) in each of the four compass directions.*
- *Unfortunately the robot's navigational system is broken, so when it executes a 'Move' action, it moves randomly to one of the adjacent squares.*
- *The robot's task is to determine its current location.*

- Known map, unknown initial location



- First observation: blocked in N,S,W
  - Where am I?

Example 2: robot localization



(a) Possible locations of robot after E_1 = NSW

b: set of all locations

$\Downarrow$ $NESW = 1011$ (obstacles in *N*, *S*, *W*)

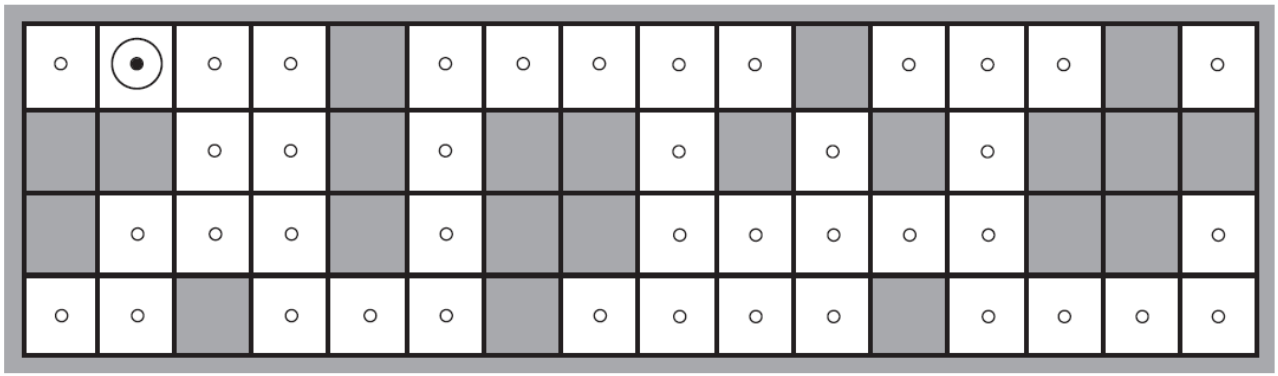
$b_0 = \text{UPDATE}(b, 1011)$ // only four possible locations

- One of four possible locations

**The move make the new belief state contains all the locations that are one step away from previous one.*
**When the second percept, NS, arrives, the robot find that the belief state has collapsed down to the single location.*

- Make another move (in unknown direction)
- Second observation: N,S
  - Where am I?

Example 2: robot localization



(b) Possible locations of robot After $E_1 = NSW, E_2 = NS$

Next, the robot executes a *Right* action.

b_a= PREDICT$(b_0, Right)$

⇩ $NESW = 1010$

b_1 =Update$(b_a, 1010)$ // One possible location!

- Location (=state) is narrowed down to just one location
- For recursive state estimation in general,
  - Action increases uncertainty due to nondeterministic environment
  - Observation, partial or full, decreases uncertainty
  - Uncertainty will repeatedly grow then shrink as search progresses